

this variable with a mutex `sum_mutex`. Each thread tries to acquire the `sum_mutex` when it wants to update the variable `Sum`. Hence, as the number of threads increases, the lock contention on `sum_mutex` also increases, leading to a scalability bottleneck.

As the number of processors increases, the situation actually becomes worse. Consider a 32 multiprocessor cache-coherent NUMA system on which this application is run. As each thread tries to acquire the lock from a different processor, the cache line containing the lock ping-pongs across the caches, leading to long memory latencies. A similar issue occurs with the global shared variable `Sum`. However, note that it is possible for each thread to maintain a private `Sum` variable per thread, which holds the summation of the elements in the array section assigned to it. After all threads finish their calculations, the main thread can read each thread's private `sum` variable and use it to update the global `sum` variable. This will prevent the global shared variable `Sum` from becoming the contention bottleneck. Given below is a code snippet which illustrates this:

```
#include <pthread.h>
pthread_t* Threads[MAX_THREADS];
int thread_private_sums[MAX_THREADS];
int Array[NUM_ELEMENTS];
int Sum;

void SumArray(int chunk_size)
{
    long tid = pthread_self();
    int start = tid*chunk_size;
    int stop = tid*chunk_size + chunk_size;

    for (int i = start; i < stop; i++)
    {
        thread_private_sums[tid] += Array[i];
    }
}

int foo(int num_threads)
{
    for (int i = 0; i < num_threads; i++)
        pthread_create(&Threads[i], NULL, SumArray, chunk_sz);

    for (int i = 0; i < num_threads; i++)
        Sum += thread_private_sums[i];

    printf("Final Sum is %d\n", Sum);
}
```

Note that we no longer need to lock the variable `Sum`, since it is updated only by the main thread.

In this particular example, there is no data communicated across threads, and hence the earlier lock-contention bottleneck on the variable `Sum` is a spurious bottleneck introduced by the programmer. We removed this by maintaining thread-private sums which were later combined to give the correct result. This is an example of a *data-parallel* application,

wherein each thread operates on an independent set of data, and there is insignificant data-sharing across threads. Such data-parallel applications are more amenable to automatic parallelisation by compilers.

However, in many real-life applications, there is considerable data communication across threads during the lifetime of the application. The data shared across threads become a scalability bottleneck, due to the communication and synchronisation costs associated with them. Consider a simple producer-consumer pattern, wherein one thread produces the data that is consumed by another thread—for example, one thread writes messages into a queue, which is processed by another thread. Such applications are known as a ‘producer-consumer’ class of applications. In this case, since the data sharing is true data sharing, we cannot avoid communication and synchronisation costs as in our previous example of data-parallel applications.

Another class of multi-threaded applications exhibits what is known as ‘migratory data’ patterns. In this case, the application consists of multiple stages, with the same data getting operated on by different threads as it passes through each stage. Take, for instance, an image-processing application with each image-processing step being a stage, and a group of threads performing the work in each stage. This form of parallelism is known as pipeline parallelism; it is limited by inter-stage dependencies, and the duration of the longest stage.

Now, coming back to our example code snippet: we have removed the lock contention on the update of the global shared variable `Sum`—hence, we expect that the code should scale well as we increase the number of processors. However, this does not happen, due to a couple of latent bugs hidden in our code snippet. I would like readers to experiment with the code snippet, and identify what are the issues that prevent it from scaling. If you have access to a 16- or 32-processor SMP system, you can run this code snippet with different numbers of threads, and find out the speed-up as the number of threads are increased. What kind of speed-ups do you get as you progress from 1 to 2, 4, 8 and 16 threads?

Takeaway problem

This month’s takeaway problem is for readers to identify the latent bugs in our code snippet that uses the thread-private sum variables. Here are a couple of clues to help you along: most multi-threaded applications suffer in performance due to false sharing. Do we have any false sharing issues in our code snippet? Also, as we increase the number of threads, thread set-up and tear-down costs also increase proportionately. So the programmer needs to keep in mind whether each thread has enough useful work to justify these costs.

Please do send me your answers to this month’s questions. If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and wishing you the very best! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools. [If you are preparing for computer science programming interviews, you may find it useful to visit Sandya’s programming interviews discussion group ‘Computer Science Interview Training (India)’ on LinkedIn (www.linkedin.com).]